Objektumelvű elemzés UML-lel

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

UML diagramjai

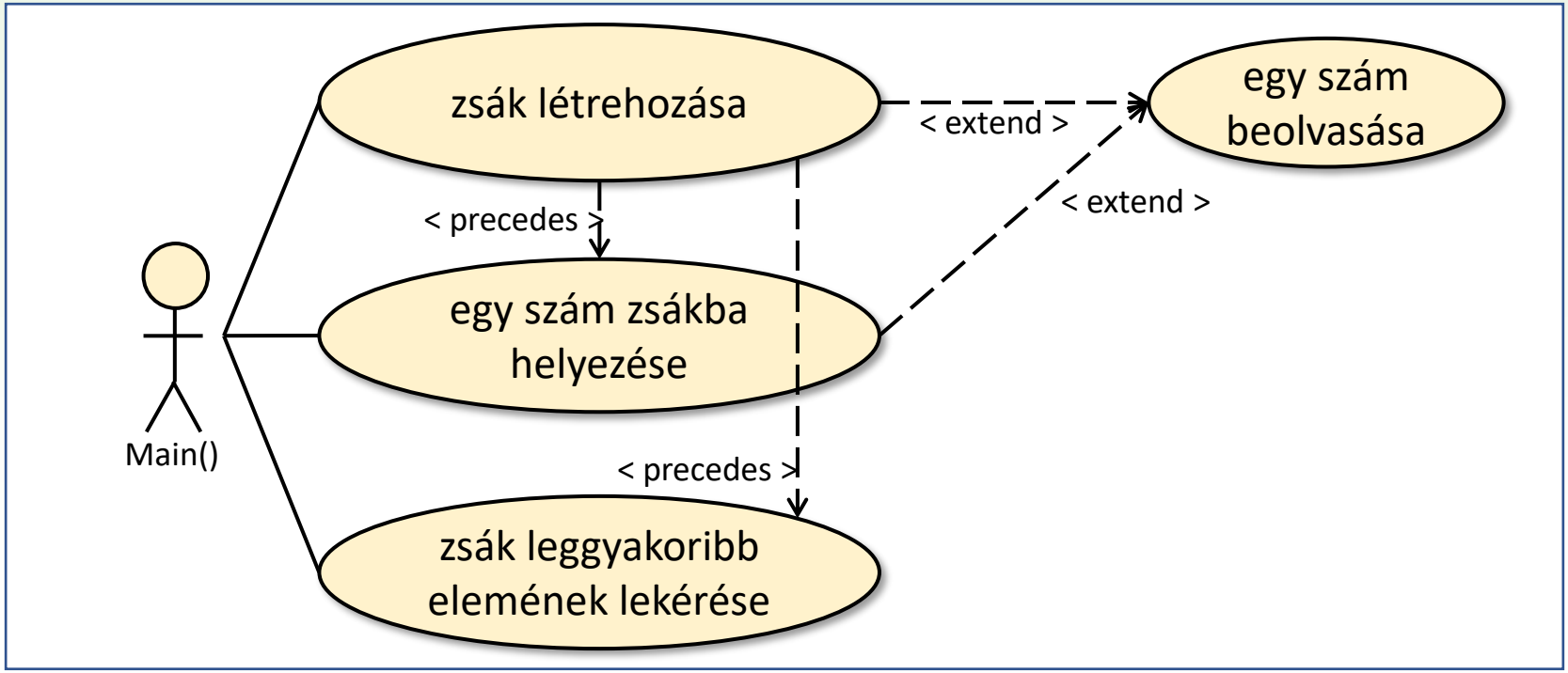
- Az UML (*Unified Modeling Language*) egy grafikus nyelv, amelyet az objektumelvű szoftverfejlesztési folyamat során használhatunk: különféle diagramokat ajánl egy probléma elemzéséhez, a megoldásának megtervezéséhez, megvalósításához, és teszteléséhez.
- Az UML diagramokat aszerint szokták csoportosítani, hogy a készülő rendszert melyik oldaláról, azaz milyen nézetből mutatják be.A legfontosabbak a:
  - szerkezeti nézet: pl. objektum-, osztály-, komponens-, kihelyezési-, csomag diagram
  - viselkedési nézet: pl. használati eset-, kommunikációs-, szekvencia-, állapotgép-, tevékenység diagram

Elemzést támogató diagramok

- A feladatok elemzéséhez az alábbi diagramokat fogjuk használni:
  - használati eset diagram
    - ez az egyetlen nem objektumelvű nézet, amely a feladat fő funkcióit írja le
  - objektum diagram
    - egy adott időpillanatban létező objektumokat és azok kapcsolatait jellemzi
  - kommunikációs diagram
    - objektumok közötti üzenetváltásokat (szignál küldés, metódus-hívás) mutatja
  - osztály diagram
    - az objektumok és a közöttük létrejövő kapcsolatok általános leírására szolgál; elsősorban inkább a tervezés és megvalósítás fázisaiban is használják
  - szekvencia diagram
    - üzenetváltások szekvenciájának egy lehetséges forgatókönyvét (szkenárióját) mutatja be, ennél fogva a tesztesetek leírására is alkalmas

Használati eset diagram

- Megmutatja, hogy a tervezett rendszer
  - milyen funkciókat lát el, milyen szolgáltatásokat biztosít
  - milyen (tartalmazási, rákövetkezési) kapcsolatoknak kell fenn állni az egyes funkciók között
  - kik számára nyújtja a szolgáltatásokat
  - milyen követelményeket vár el a környezetétől



Használati eset diagram elemei

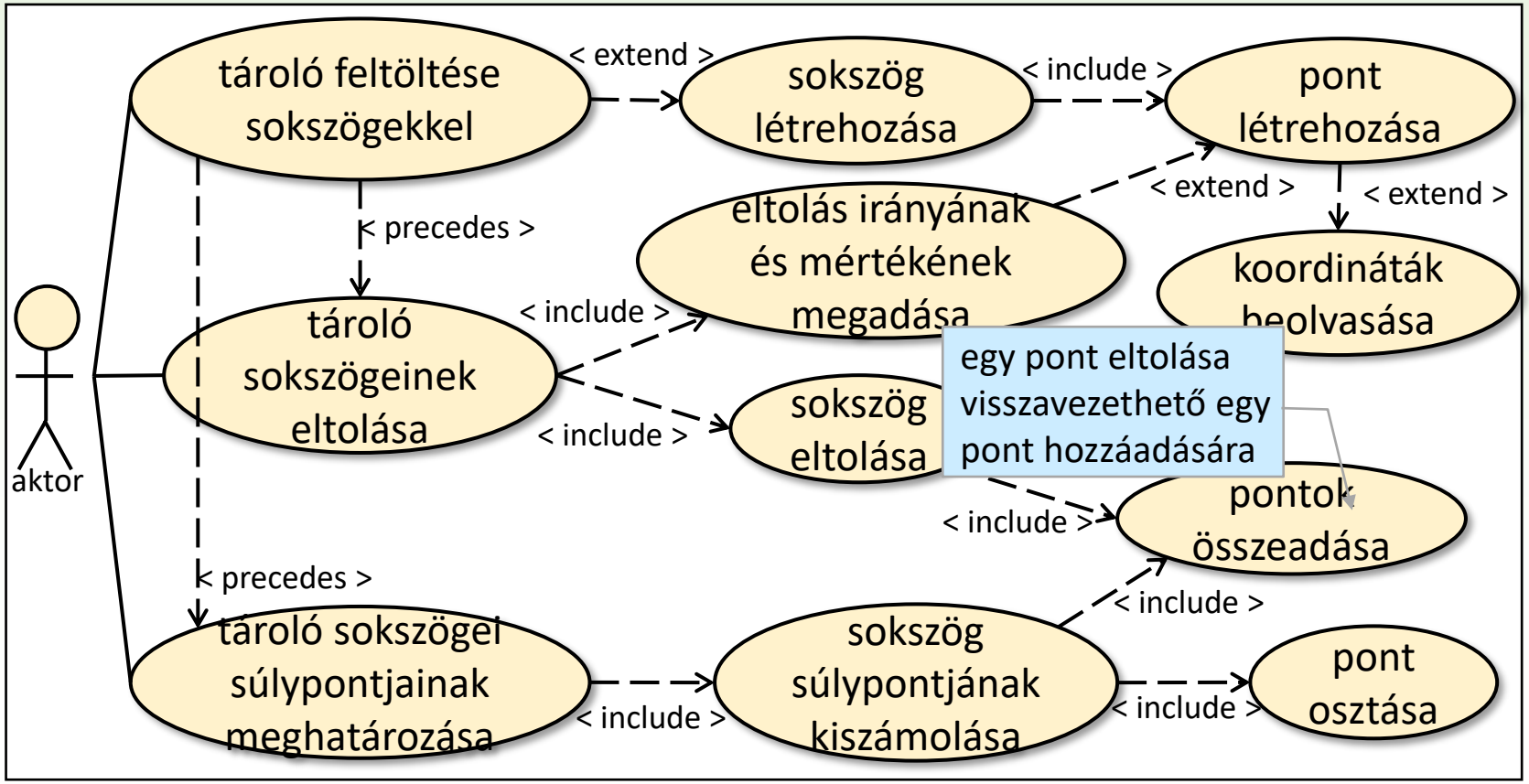
- Használati esetek: azon tevékenységek (funkciók, szolgáltatások), amelyek a rendszert kívülről szemlélő (azaz a felhasználó vagy más külső szereplő) szempontjából azonosíthatóak, illetve kezdeményezhetőek.
- Kapcsolatok a használati esetek között, amelyek a használati esetek
  - részekre bontását, illetve plusz funkciókkal történő kiegészítését ábrázolják
  - általános használati esetből történő származtatására mutatnak rá
  - sorrendjét határozzák meg
- Aktorok: a funkciókat használó felhasználók vagy más külső szereplők, akik vagy amik a tervezett rendszer működését vezérlik. Külön jelölhető az, hogy egy aktor az aktoroknak mely csoportjához tartozik (specializálás), illetve az, ha egyféle aktorból hány lehet (multiplicitás).

Használati esetek kapcsolatai

- Használati eset részletezése
  - include: rámutat egy használati eset jól elkülöníthető, önállóan is kezdeményezhető olyan részére, amely nélkül a tartalmazó tevékenység már nem teljes (hanem absztrakt)
  - extend: egy önmagában is teljes (tehát nem absztrakt) használati esetet opcionálisan kiegészítő másik esetre mutat
- Használati esetek közötti származtatás
  - amikor egy általánosan megfogalmazott használati esetnek több eltérő konkrét változata is beazonosítható
- Használati esetek rákövetkezési sorrendje
  - precedes: két, az aktor által közvetlenül kezdeményezhető tevékenység közötti sorrendet jelöli. (Célja, hogy a későbbi tevékenységet vagy ne lehessen addig kezdeményezni, amíg a megelőző tevékenységet nem hajtották végre, vagy jelezze annak hiányát)
  - invokes: rámutat arra, hogy egy tevékenység végrehajtását mely másik tevékenységnek kell követnie (az aktor akaratától függetlenül)

Példa

Töltsünk fel egy tárolót különféle sokszögekkel, és mindegyiket toljuk el ugyanazon irányba és mértékkel, majd számoljuk ki az így nyert sokszögek súlypontjait. A csúcspontok és súlypontok koordinátái, sőt az eltolást leíró helyvektor végpontjának koordinátái is legyenek egész számok.



Felhasználói történet (user story)

- A használati eset diagram önmagában nem ad elégséges képet a megvalósítandó rendszerről, hiszen az egyes tevékenységekről a nevükön kívül nem árul el túl sokat.
- A felhasználói esetek felhasználói csoportonként („AS a …”) történő táblázatos („user story”) leírásában viszont részletesen ki lehet fejteni a felhasználói tevékenységeket:
  - mi a tevékenység neve,
  - milyen előfeltétel meglétét feltételezi (GIVEN)
  - milyen esemény hatására következik be (WHEN)
  - mi a hatása a végrehajtásának, milyen eredményt ad (THEN).

AS a …

eset		leírás
tevékenység neve	GIVEN	tevékenység kiváltásakor feltételezett alaphelyzet
	WHEN	tevékenység kiváltása
	THEN	tevékenység hatása

Példa

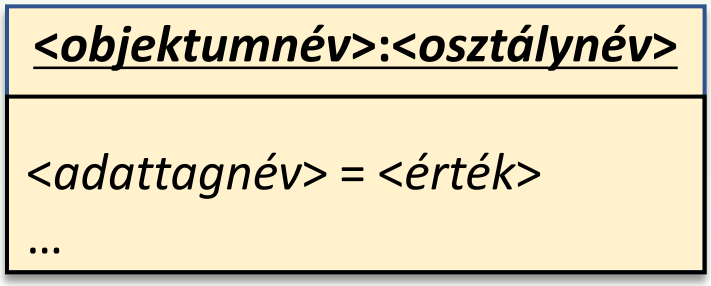
eset		Leírás
tároló feltöltése sokszögekkel	GIVEN	sokszögek, és azokat tárolni képes tároló (pl. sorozat)
	WHEN	sokszögek elhelyezésének kérelme
	THEN	a tároló sokszögeket tartalmaz
sokszög létrehozása	GIVEN	alkalmazás egy sokszög létrehozására vár
	WHEN	csúcsok megadása (darabszámuk és a csúcsok síkbeli pontjai)
	THEN	létrejön egy sokszög (objektum)
pont létrehozása	GIVEN	síkbeli pont koordinátái
	WHEN	síkbeli pont létrehozásának kérelme
	THEN	létrejön egy síkbeli pont (objektum)
tároló sokszögeinek eltolása	GIVEN	tároló a sokszögekkel, az eltolás helyvektorának végpontja
	WHEN	sokszögek eltolásának kérelme és adott
	THEN	A tároló sokszögeit egymás után eltolja a megadott vektorral
eltolás irányának és mértékének megadása	GIVEN	alkalmazás az eltolás helyvektora vépontjának létrehozására vár
	WHEN	síkbeli pont koordinátáinak megadása
	THEN	egy síkbeli pont létrehozása

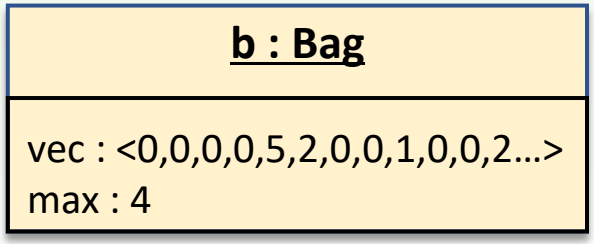
Példa

eset		leírás
tároló sokszögei súlypontjainak meghatározása	GIVEN	tároló sokszögekkel
	WHEN	súlypontok meghatározásának kérelme
	THEN	a tároló sokszögeinek egymás után kiszámolja és kiírja a súlypontját
sokszög súlypontjának kiszámolása	GIVEN	sokszög
	WHEN	súlypont számítás igénye
	THEN	sokszög csúcspontjai alapján létrejön egy síkbeli pont (objektum)
sokszög eltolása	GIVEN	sokszög és az eltolás helyvektorának végpontja
	WHEN	eltolás igénye
	THEN	módosulnak a sokszög csúcspontjai
pontok összeadása	GIVEN	két síkbeli pont
	WHEN	összeadás igénye
	THEN	az egyik pont koordinátái két másik pont koordinátáinak összege
pont osztása	GIVEN	síkbeli pont és egy egész szám
	WHEN	osztás igénye
	THEN	a pont koordinátái az eredeti koordináták adott számmal vett hányadosai

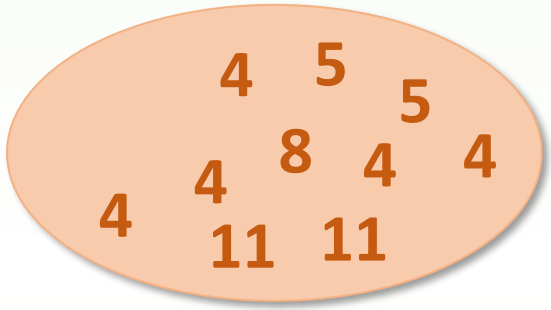
Objektum diagram

- Az objektum diagram a tervezett rendszer működésének egy adott időpillanatában létező objektumok (a populáció) leírására szolgál.
- Egy objektumot meghatároz
  - az osztálya, amely az ugyanolyan adattagokkal és metódusokkal rendelkező objektumokat jellemzi.
  - a neve (amit nem kötelező megadni),
  - az állapota (amit az adattagjainak értékei jelölnek ki).





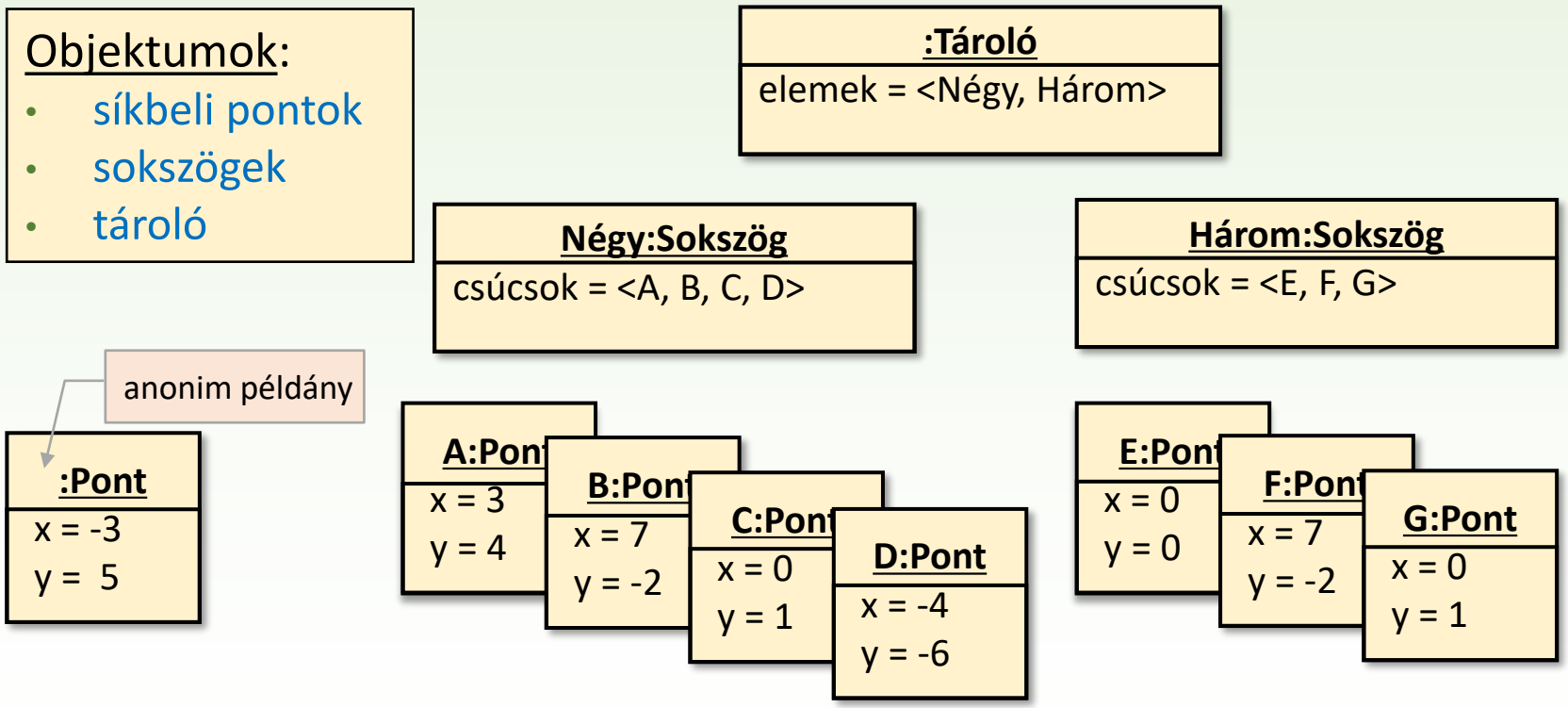
Az objektum diagram nem tartalmazza az objektumra meghívható metódusokat.



Példa

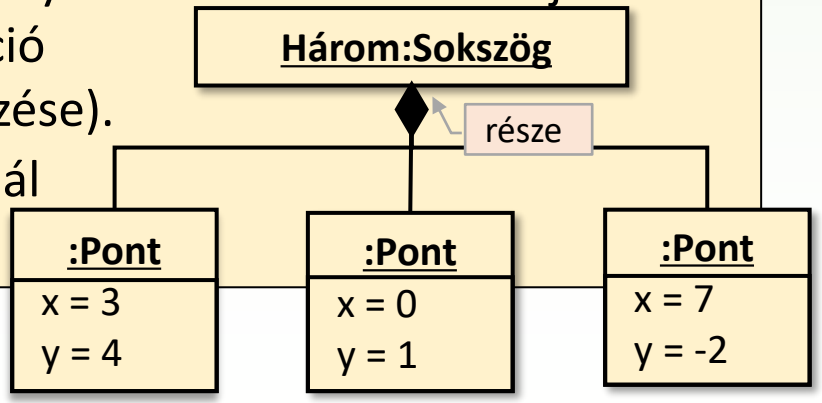
Töltsünk fel egy tárolót különféle sokszögekkel, és mindegyiket toljuk el ugyanazon irányba és mértékkel, majd számoljuk ki az így nyert sokszögek súlypontjait. A csúcspontok és súlypontok koordinátái, sőt az eltolást leíró helyvektor végpontjának koordinátái is legyenek egész számok.

Objektumok:
- síkbeli pontok
- sokszögek
- tároló



Objektumok kapcsolatai

- Az objektumok között ideiglenesen vagy hosszabb távon alakulhatnak ki kapcsolatok.
  - objektum adattagjai hivatkozhatnak más objektumokra (például azokra, amelyekből felépül, amelyek a részei)
  - metódusa hívhatja egy másik objektum metódusát, de ehhez ismernie kell a másik objektumot, azaz tudnia kell rá hivatkozni: a másik objektum hivatkozását az egyik adattagjában tárolja, vagy megkapja paraméterként
  - küldhet a másik objektumnak egy jelzést (szignált)
- Két objektum közötti kapcsolatot egy folytonos összekötő vonal jelez. Ennek végén lehet nyíl (a kommunikáció iránya), vagy rombusz (tartalmazás jelzése).
- A kapcsolatokat az osztály diagramoknál fogjuk részletesen tárgyalni.



Objektumok felelősségi köre

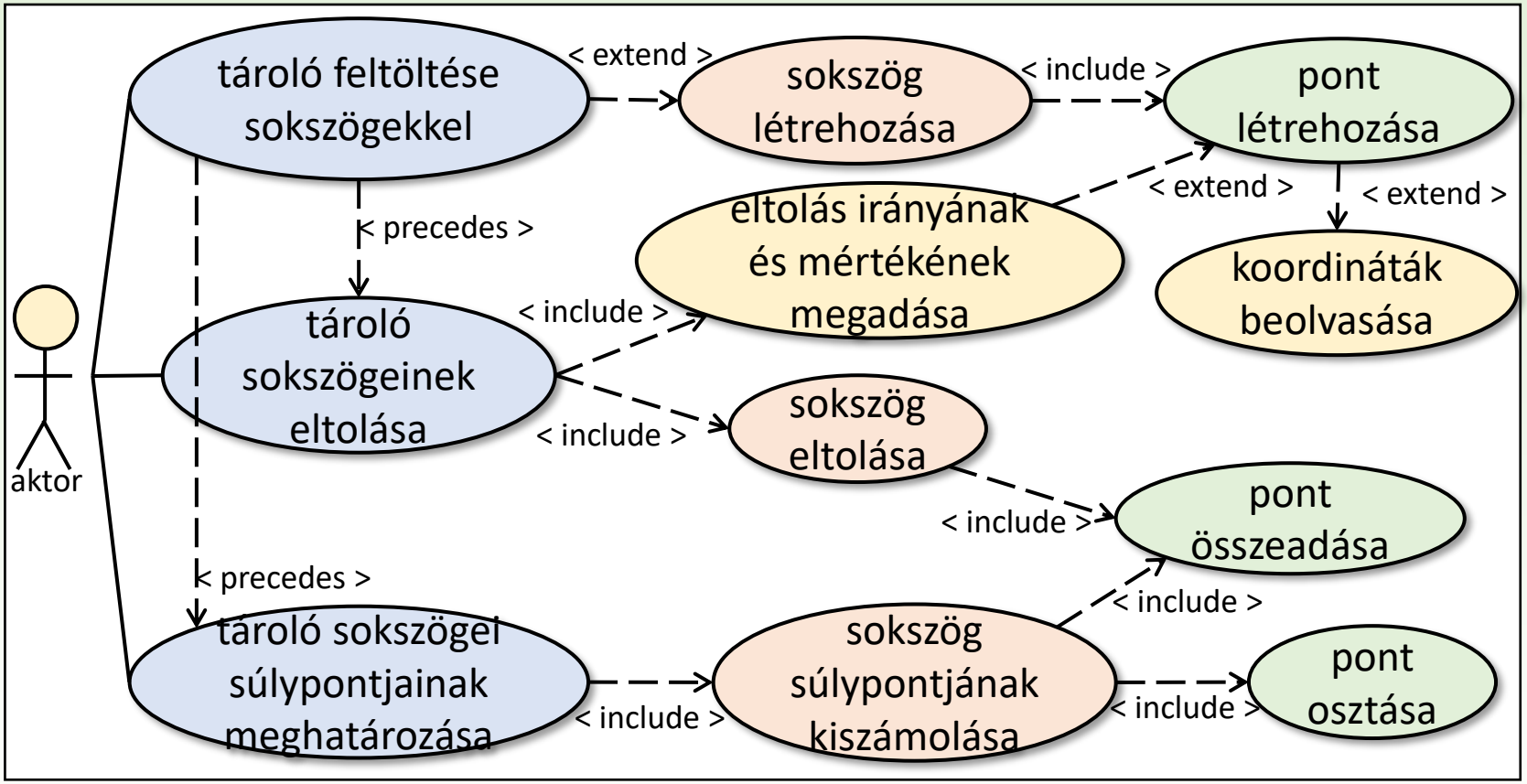
- Egy objektumelvű megoldásban a részfeladatokat megoldó részprogramokat az egyes objektumokhoz rendeljük, ezek lesznek az objektum metódusai. Egy metódus ahhoz az objektumhoz (annak a felelősségi köréhez) tartozik, amelynek az adattagjaival végez műveletet.

Single responsibility
O
L
I
D

Egy objektum – egy felelősségi kör.

- Ne tervezzünk ”diktátor” objektumot, amely több felelősségi kör metódusait gyűjti egybe.
- Ne osszunk meg egy felelősségi kört több objektum között.

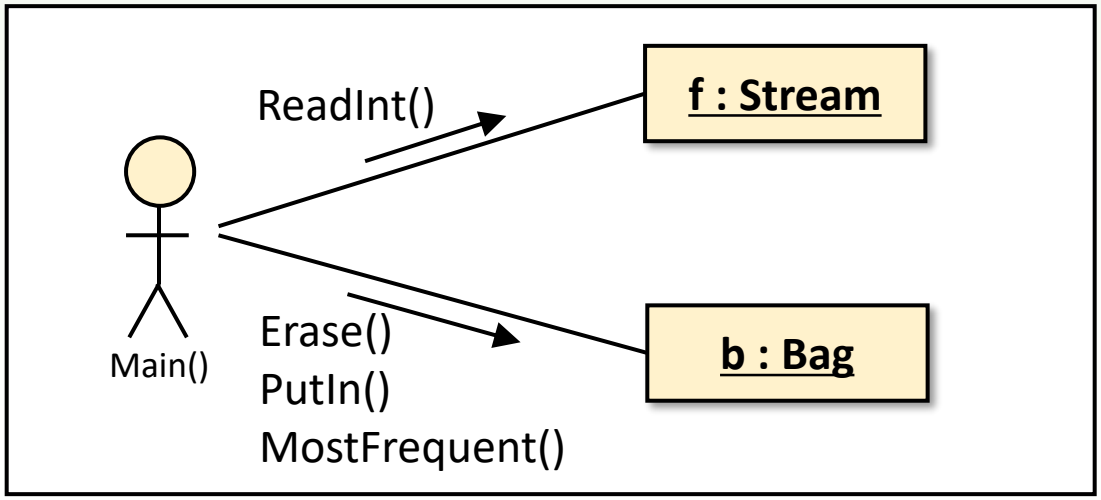
Példa objektumainak felelőssége



- sokszög: létrehozás, eltolás, súlypont kiszámítás
- síkbeli pont: létrehozás, összeadás, osztás
- tároló (sorozat): elem elhelyezése (hozzáfűzése),
  elem kinyerése (hossz lekérdezése, adott indexű elem)

Kommunikációs diagram

- A kommunikációs diagram azt mutatja meg, hogy az egymással kapcsolatba kerülő objektumok milyen üzeneteket küldenek egymásnak (azaz egy objektum mely metódusát hívja meg egy másik objektumnak, vagy milyen szignált küld neki).
- Hasonlít egy objektum diagramhoz, de nem tünteti fel az objektumok adattagjait, és azok értékét, azaz állapotát, ennél fogva általánosabb, ugyanakkor megmutatja, hogy az objektumok között milyen irányban, milyen tartalmú üzenet valósulhat meg. Ez által leírja az objektumok felelősségi körét, azaz a metódusait is.

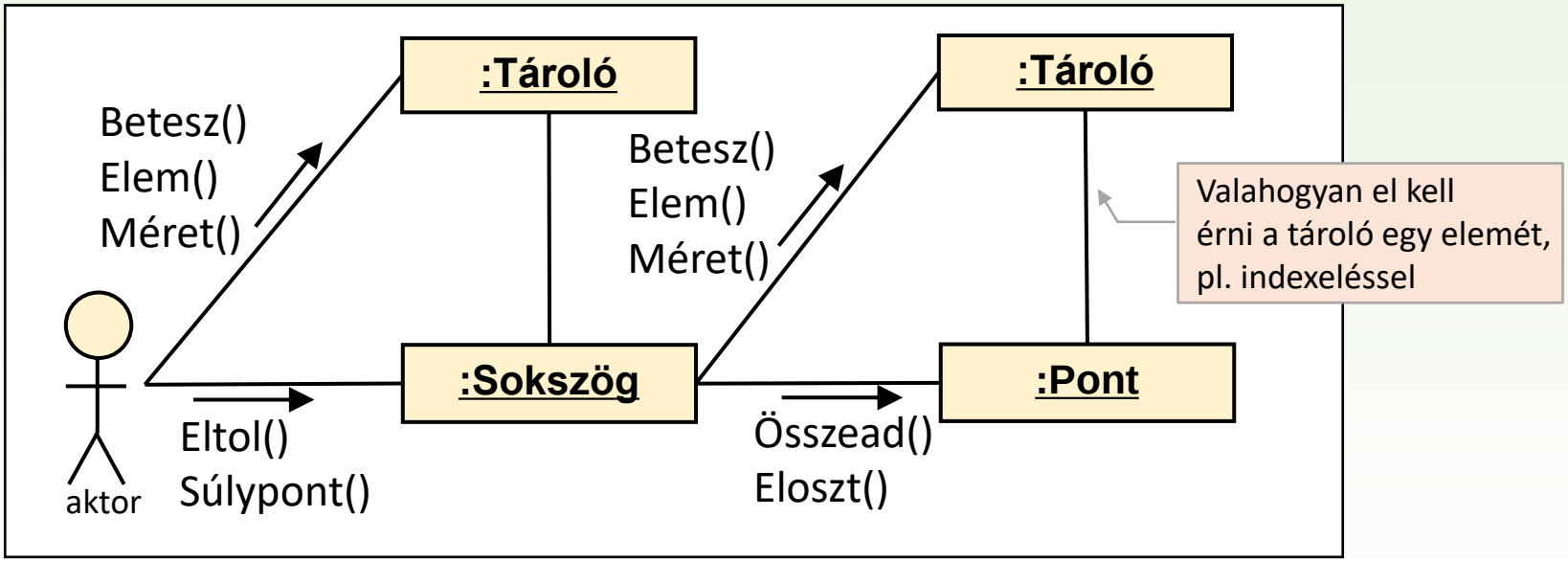


Kommunikációs diagram üzenetei

- Üzenet az, amikor
  - a küldő objektum meghívja a fogadó objektum egyik metódusát,
  - a küldő objektum szignált küld a fogadó objektumnak.
- Az üzenetküldés módja lehet
  - szinkron: a küldő objektum megvárja, amíg a fogadó feldolgozza az üzenetét, és csak ezen feldolgozás esetleges eredményét megkapva folytatja a saját tevékenységét, 
  - aszinkron: az üzenet elküldése után a küldő és a fogadó objektumok tevékenységei párhuzamosan folynak. 
- Egyéb jelölések
  - Kijelölhető az üzenetek sorrendje (azok sorszámozásával).
  - Megadható az üzenetek előfeltétele (szögletes zárójelpár között).
  - Jelölhető, ha egy üzenet ciklikusan ismétlődik.

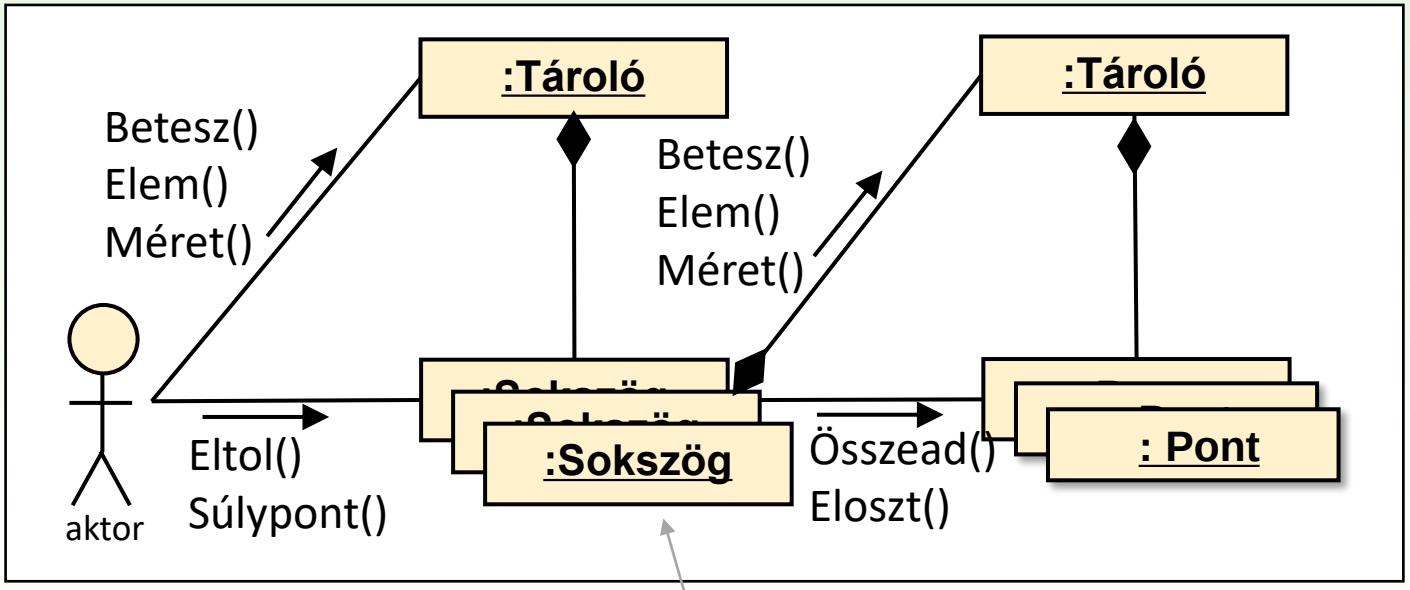
Példa

Töltsünk fel egy tárolót különféle sokszögekkel, és mindegyiket toljuk el ugyanazon irányba és mértékkel, majd számoljuk ki az így nyert sokszögek súlypontjait. A csúcspontok és súlypontok koordinátái, sőt az eltolást leíró helyvektor végpontjának koordinátái is legyenek egész számok.



Objektum+kommunikációs diagram

- Habár nem szabványos, de lehetőség van az objektum diagram és a kommunikációs diagram közös ábrázolására is.



Osztály diagram

- Az osztály diagram a tervezett rendszer működése során létrejövő populációk általános leírására szolgál: egy osztály diagramnak számos objektum diagramot feleltethetünk meg. Az osztály diagram az objektum diagram absztrakciója.
  - Egy osztály a belőle példányosítható objektumok adattagjait és metódusait írja le.
  - Az osztály diagram írja le az objektumok közötti kapcsolatokat is.
- Elsősorban a tervezés és az implementálás eszköze, de kezdetleges formában már az elemzés során is alkalmazható : ilyenkor az adattagokról és a metódusokról még nem rendelkezünk pontos ismeretekkel (sokszor csak az osztály nevét adjuk meg).

Osztály diagram elemei

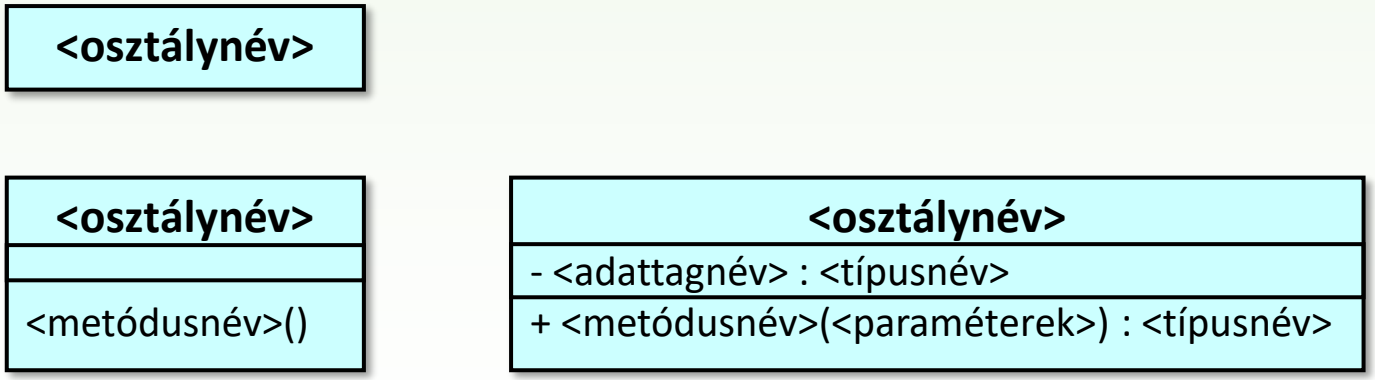
- Az osztály az azonos típusú (ugyanolyan adattagokkal és metódusokkal rendelkező) objektumok típusa. A diagramja ezért rendelkezik
  - névvel,
  - adattagokkal, (tulajdonság, attribútum, mező)
  - metódusokkal (művelet, tagfüggvény)
- Az adattagok és metódusok láthatósága egyenként beállítható
  - kívülről is látható, azaz publikus (*public* +)
  - külvilág elől rejtett: privát (*private* -) vagy védett (*protected* #)

<osztálynév>
<+|-|#> <adattagnév> : <típus> …
<+|-|#> <metódusnév>(<paraméterek>) : <típus> …

Bag
- vec : int[0..m] - max : int
+ Erase() : void + PutIn(e:int) : void + MostFrequent() : int

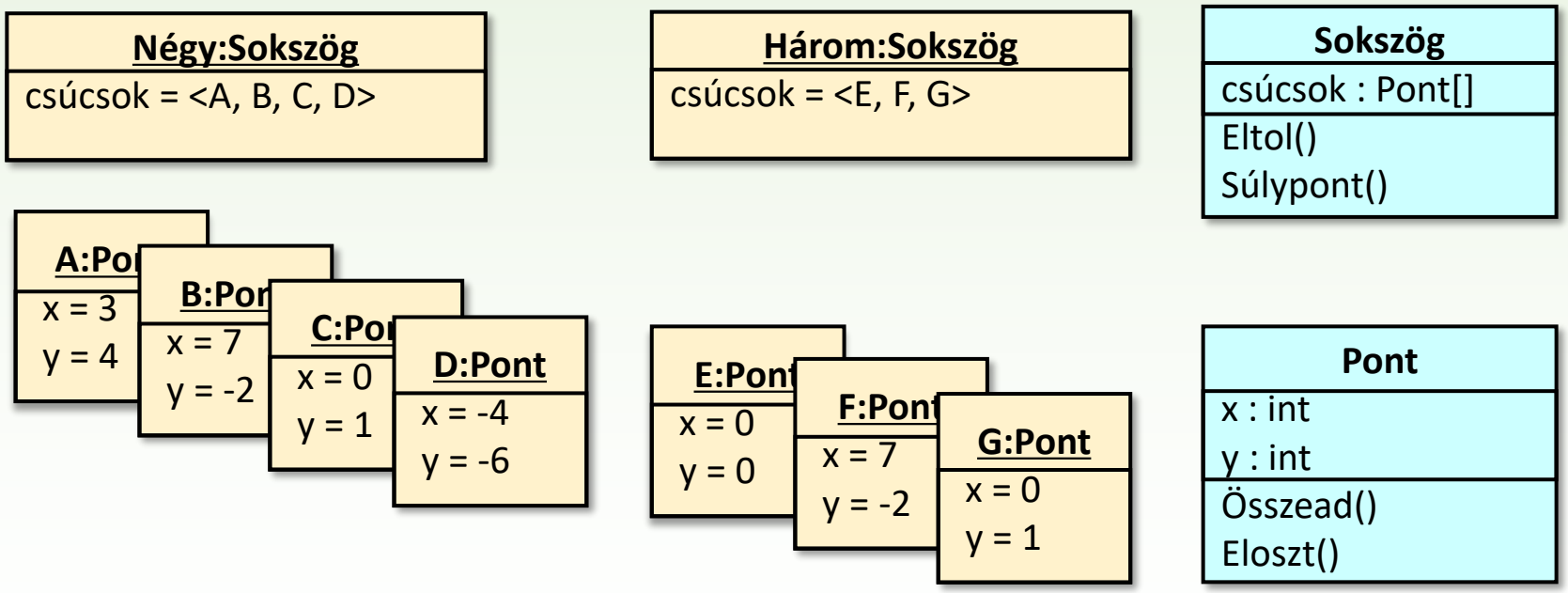
Osztály diagram részletezettsége

- Az osztály-leírás a modellezés során fokozatosan alakul ki, ezért a modellezés adott szintjén bizonyos részletei még hiányozhatnak.
  - Hiányozhatnak belőle az attribútumok és/vagy a metódusok.
  - Hiányozhat az adattagok típusa, a metódusok paraméterezése és a visszatérési típusa.
  - Hiányozhat a láthatósági jelölések feltüntetése.



Példa

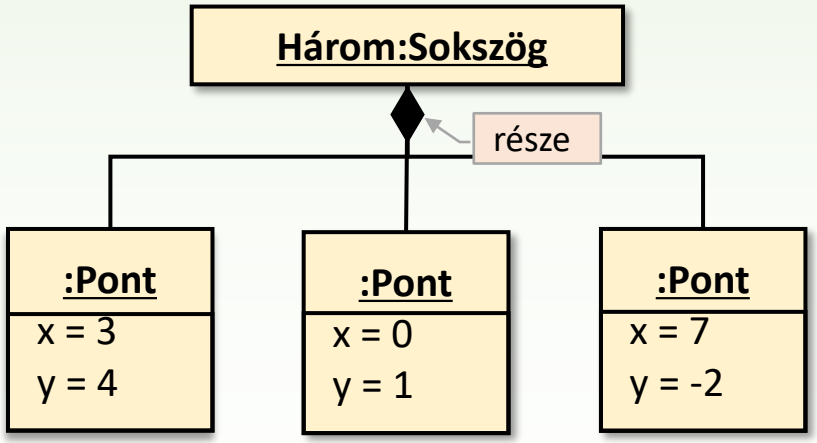
Töltsünk fel egy tárolót különféle sokszögekkel, és mindegyiket toljuk el ugyanazon irányba és mértékkel, majd számoljuk ki az így nyert sokszögek súlypontjait. A csúcspontok és súlypontok koordinátái, sőt az eltolást leíró helyvektor végpontjának koordinátái is legyenek egész számok.



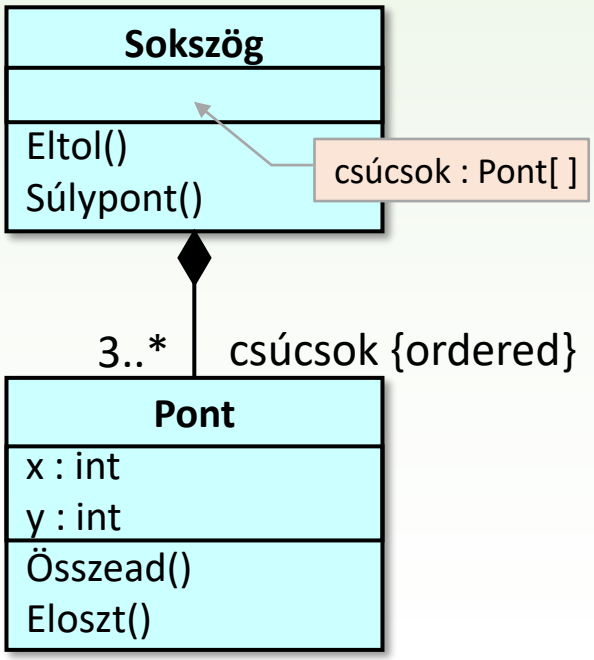
Osztály diagramban ábrázolt objektum-kapcsolatok

- Az objektumok közötti kapcsolatok osztályszintű leírását egy későbbi előadásban részletesen tárgyaljuk.

Objektumok közötti kapcsolatok ábrázolása objektum diagramban:

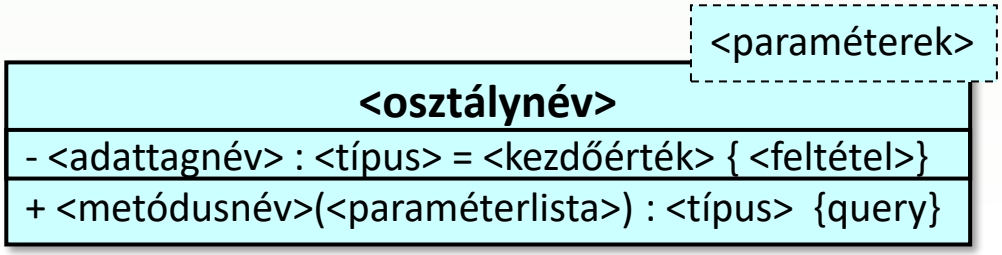


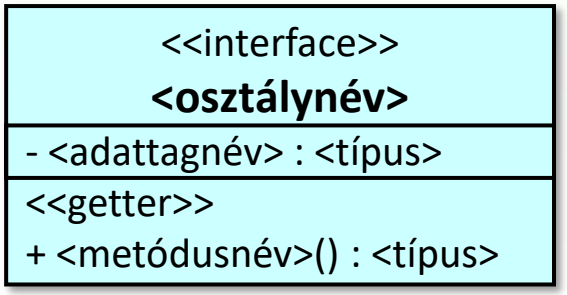
Objektumok közötti kapcsolatok ábrázolása osztály diagramban:



Osztály diagram kiegészítései

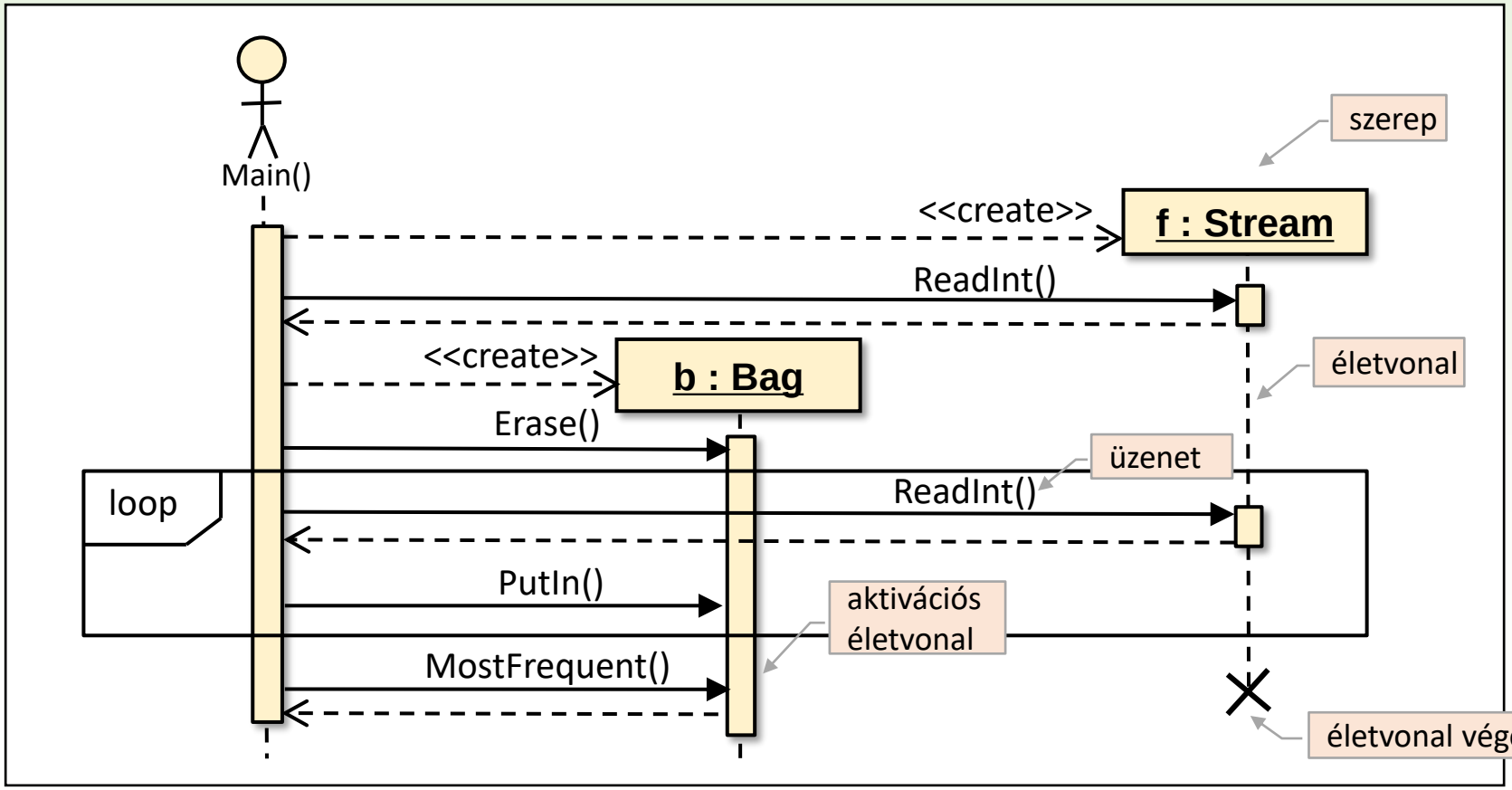
- Az osztályok a példányaik jellegzetes viselkedése alapján különféle kategóriákba (sztereotípiákba) sorolhatók, amelyet <<…>> jelzés között írhatunk le (pl. <<interface>> , <<enumeration>>).
- Egy osztályt általánosítva is felírhatunk (generikus-, sablon-osztály), ha paramétereket (pl. típust helyettesítésére) vezetünk be a leírásában.
- Az adattagokhoz (a '=' jel után) kezdőértéket rendelhetünk, amiket a konstruktor állít majd be, és egy {…} jelzésben megszorításokat (típus invariáns) is tehetünk rájuk.
- A metódusok sajátos tulajdonságai (pl. query, override, virtual) is megadható {…} jelzés között. A <<getter>> az adattagok értékét lekérdező műveletek csoportját, a <<setter>> az adattagok értékét felülíró műveletek csoportját vezeti be.





Szekvencia diagram

- Az objektumok közötti üzenetváltások időbeli sorrendjét mutatja meg.
- Ez a diagram a rendszer vagy a rendszer egy részének egy lehetséges működését írja le, ezért több is szokott készülni belőle.

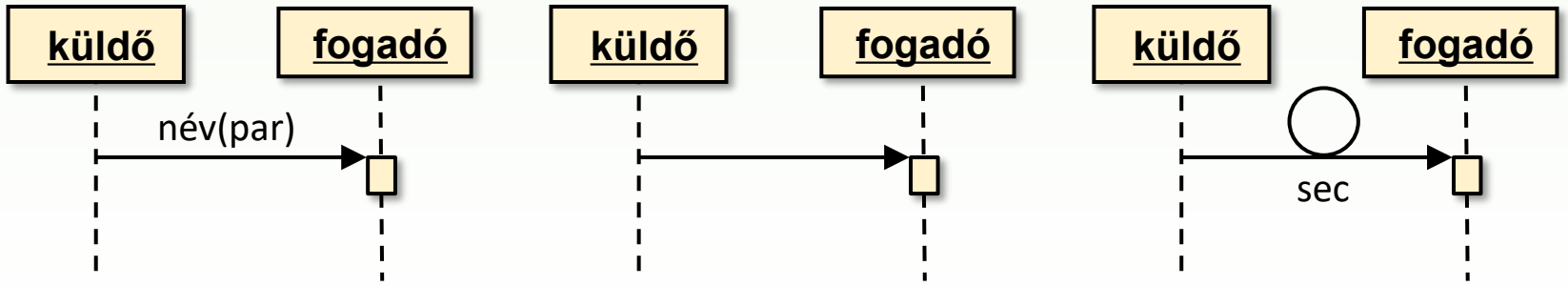


Szekvencia diagram elemei

- Osztályszerep : egy vagy akár több osztály egy vagy több objektumát testesíti meg, sokszor csak egyetlen objektumot.
- Osztályszerep életvonala (szerepből kiinduló függőleges szaggatott vonal) : egy osztályszerep létezését ábrázoló idővonal.
- Osztályszerep aktivációs életvonala (egy hosszú elnyújtott téglalap az életvonalon) : egy osztályszerepnek azon időszaka, amelyben az osztályszerepet megtestesítő objektumok más objektumok vezérlése alatt állnak, azaz egy vagy több metódusuk végrehajtás alatt áll.
- Üzenet (általában vízszintes nyíl az életvonalak között) : az objektumok közötti információ átadás formája. Megmutatja, hogy egy objektum egy üzenet elküldésével mikor hoz működésbe (akció) egy másik objektumot. (pl. meghívja annak egy metódusát.)
  Az üzenetek egymás alatti megjelenése azok időbeli sorrendjére utal. Feltüntethető két üzenet között eltelt idő is.

Szinkron üzenetek

- Olyan üzenet, amikor a küldő objektum átadja a vezérlést a fogadó objektumnak, és a saját tevékenységét mindaddig blokkolja, amíg a fogadó objektum ezt nem oldja fel. Ez lehet
  - a fogadó objektum egy metódusának hívása. Ilyenkor az üzenet címkéje a meghívott metódus neve az esetleges paraméterekkel.
  - egy szinkronizációs üzenet, amely során a küldő objektum várakozik a fogadó objektum visszajelzésére, hogy fogadta az üzenetet.
  - időhöz kötött várakozó üzenet, amikor a küldő objektum megjelölt ideig várakozik a fogadó objektum visszajelzésére.



Aszinkron üzenetek

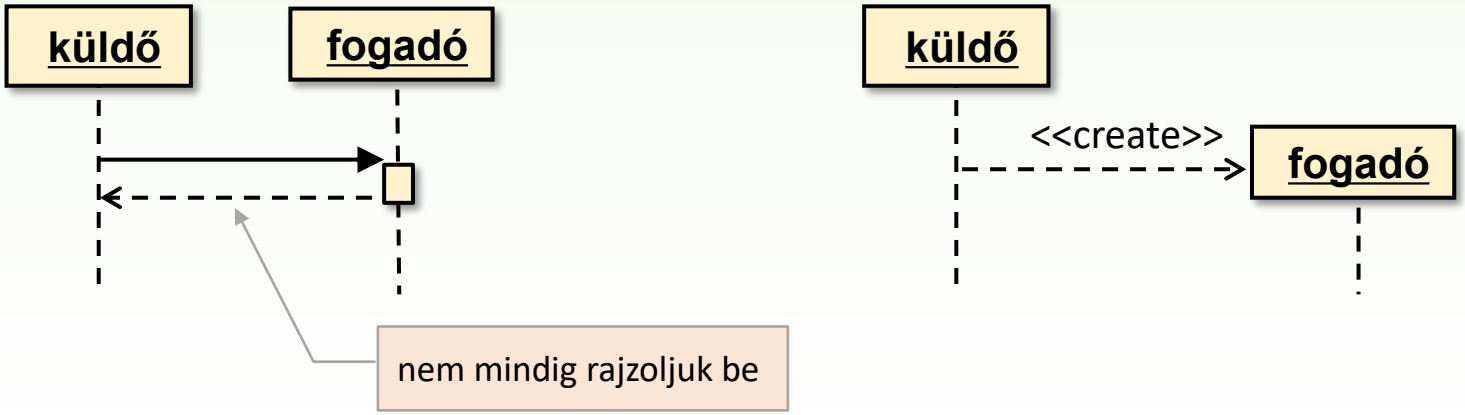
régebben:

- Olyan üzenet, amely során a küldő objektum nem szakítja meg a saját tevékenységét. Ez lehet a fogadó objektum számára küldött
  - szignál (ami kiegészülhet paraméterekkel) vagy egy aszinkron (másik szálon indított) metódus-hívás.
  - randevú üzenet, amely során a küldő objektum megjelölt ideig várakozik arra, hogy a fogadó objektum fogadja az üzenetét.



Speciális üzenetek

- Ezek azok az üzenetek, amelyeket gyakran fel sem tüntetünk a könnyebb áttekinthetőség érdekében:
  - visszatérési üzenet, amelyet a fogadó objektum küld el az őt aktiváló objektumnak egy korábban attól kapott üzenetre válaszként. Ha ez a korábbi üzenet szinkron üzenet volt, akkor a visszatérési üzenet szünteti meg a küldő objektum blokkolását.
  - példányosító (create) üzenet: létrehoz egy új osztályszerepet

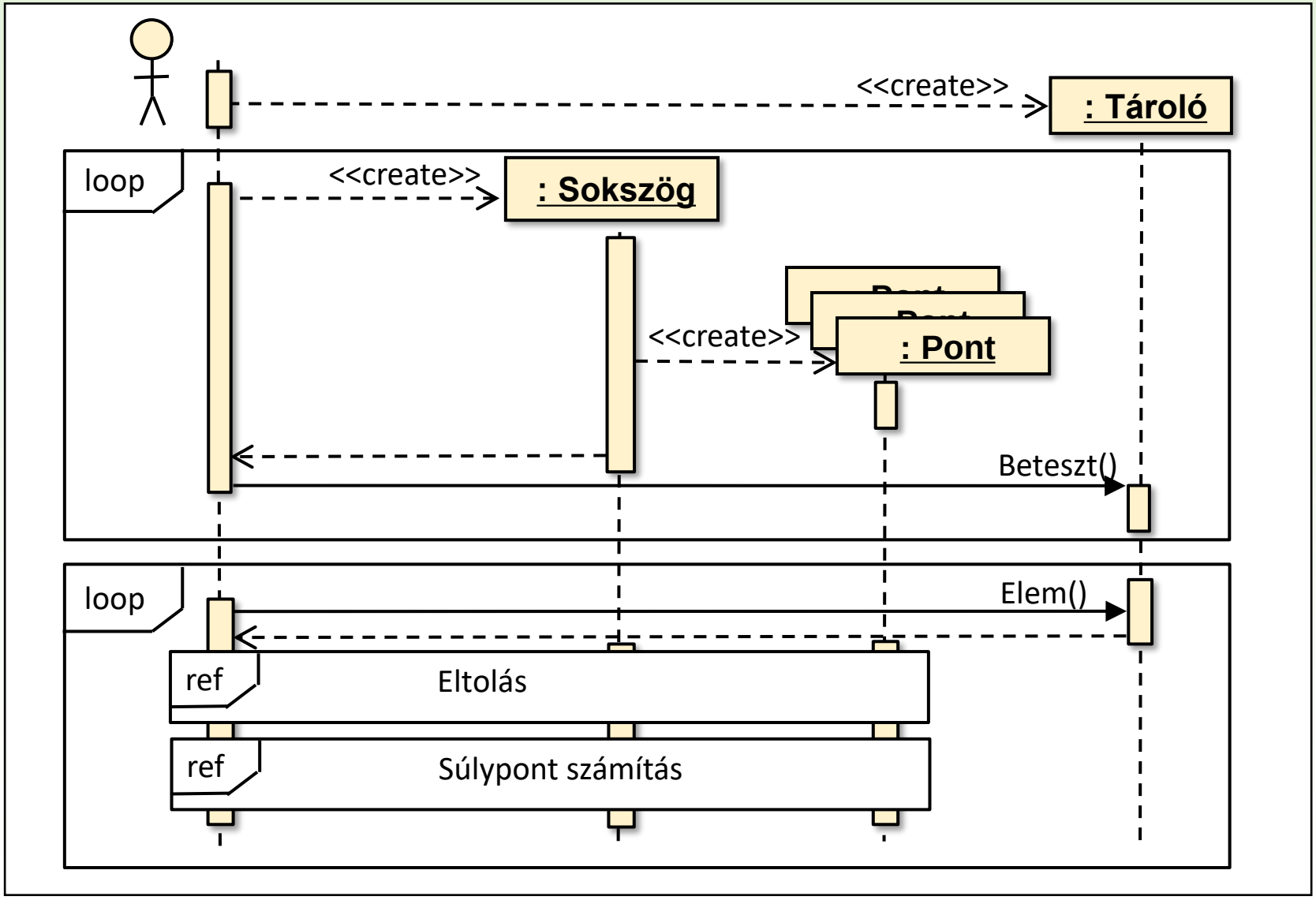


Szekvencia diagramok hierarchiája

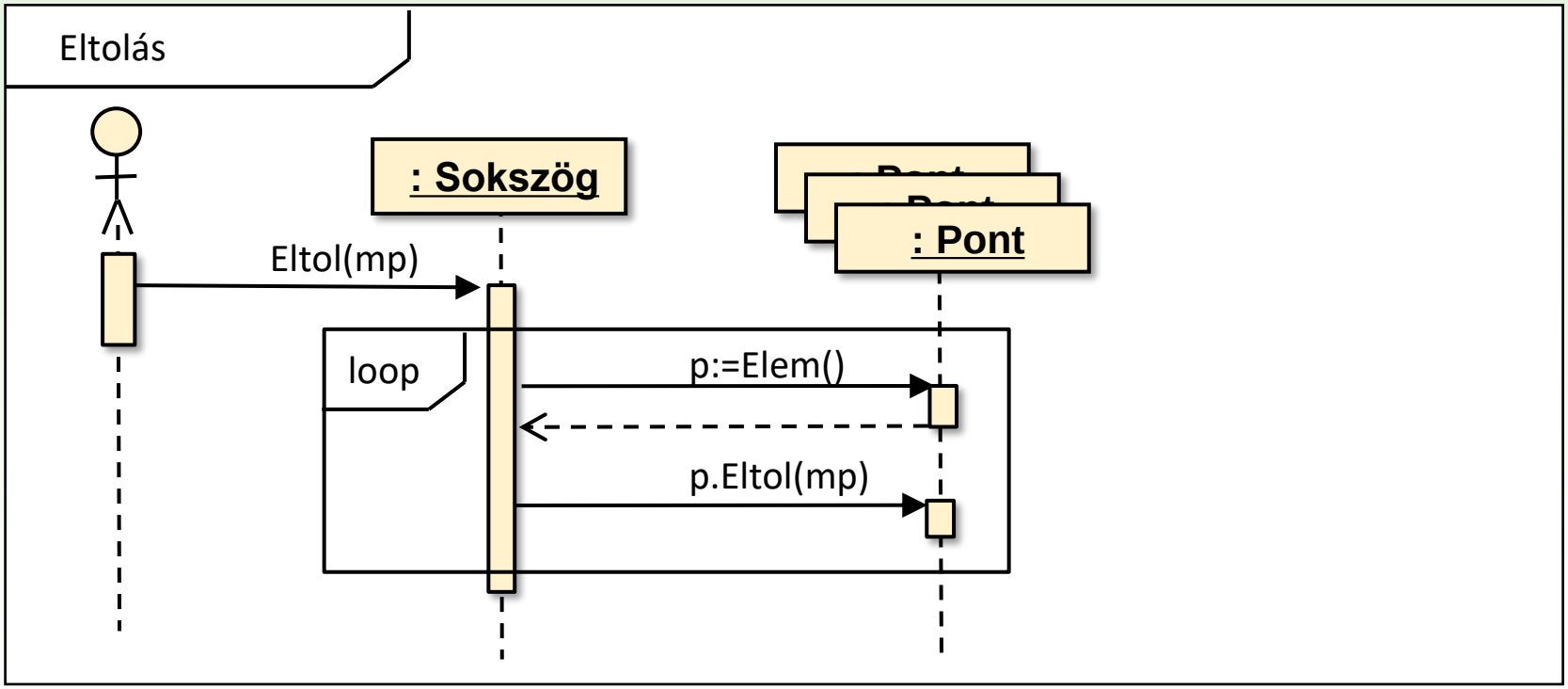
Lehetőség van arra is, hogy egy összetett szekvencia diagram bizonyos részeit az olvashatóság érdekében ideiglenesen elrejtsük, és ezeket majd külön részletezzük, finomítsuk.

- Egy szekvencia diagram egyetlen osztályszerepe elfedheti például a szerepet alkotó objektumok, objektum-csoportok közötti üzenetváltásokat. Ezeket külön szekvencia diagramban ábrázolhatjuk.
- Kiemelhető a diagram egy adott időintervallumának egy része is (ref) azért, hogy azt külön, részletesebben kifejthessük.
- Kijelölhetünk olyan részleteket (fragmentek), amelyek
  - ciklikusan ismétlődhetnek (loop)
  - adott feltétel esetén következhetnek csak be (alt, opt)
  - párhuzamosan futnak (par)
  - …

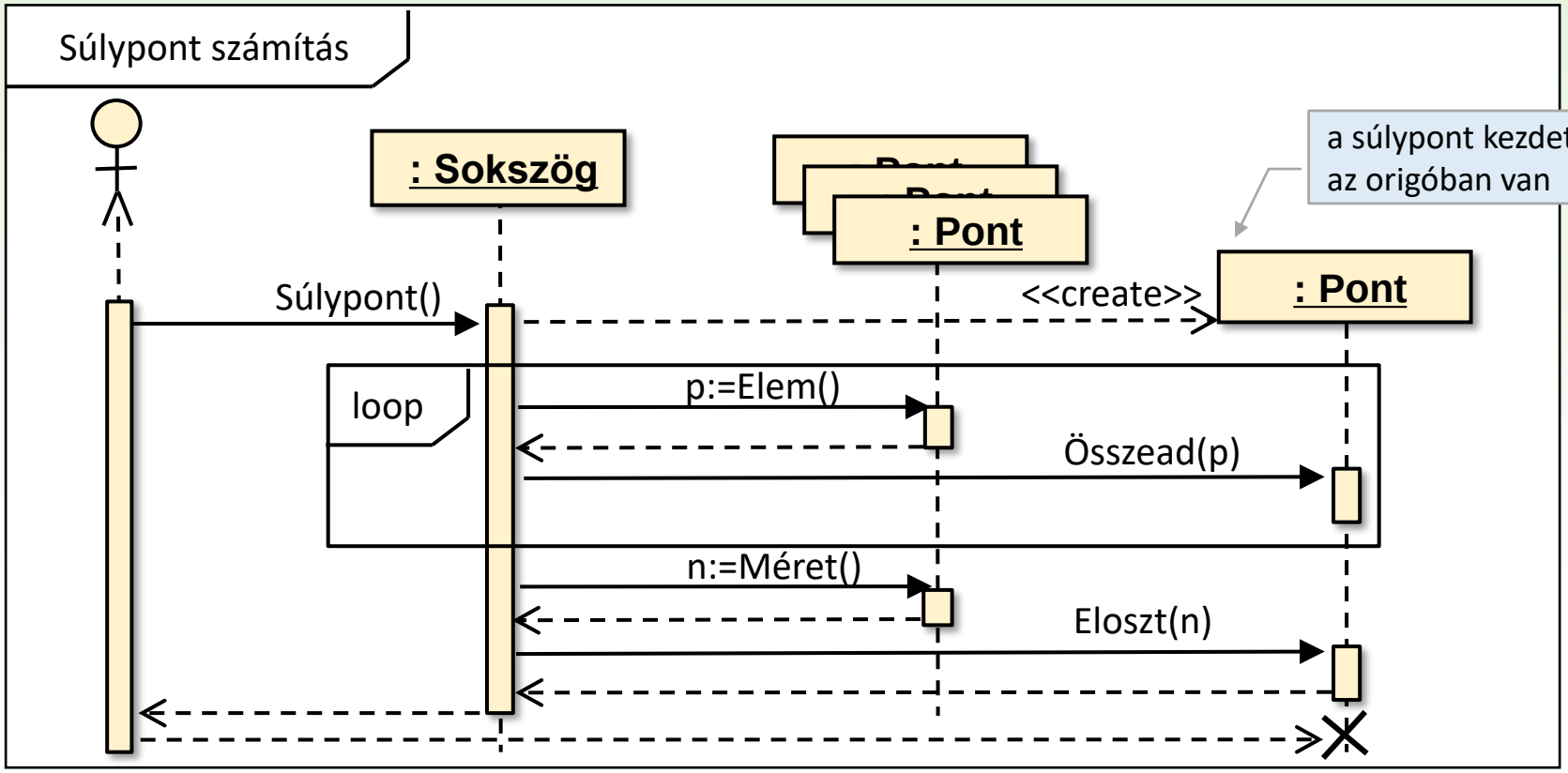
Példa


<<create>>
: Tároló
loop
<<create>>
: Sokszög
<<create>>
: Pont
Beteszt()
loop
Elem()
ref
Eltolás
ref
Súlypont számítás

Példa



Példa


Súlypont számítás
: Sokszög
: Pont
: Pont
a súlypont kezdetben az origóban van
Súlypont()
<<create>>
loop
p:=Elem()
Összead(p)
n:=Méret()
Eloszt(n)